

□ Titanic Survival Prediction - Complete ML Pipeline

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import
classification_report, f1_score, accuracy_score, confusion_matrix, roc
_curve, roc_auc_score
from xgboost import XGBClassifier
```

```
data = pd.read_csv("/kaggle/input/titanic/train.csv")
test = pd.read_csv("/kaggle/input/titanic/test.csv")
```

```
print(f"□ Training Dataset Shape: {data.shape}")
print(f"□ Test Dataset Shape: {test.shape}")
```

```
□ Training Dataset Shape: (891, 12)
□ Test Dataset Shape: (418, 11)
```

```
data.head(5)
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

		Name	Sex	Age
SibSp	\			
0		Braund, Mr. Owen Harris	male	22.0
1				
1	Cumings, Mrs. John Bradley (Florence Briggs Th...		female	38.0
1				
2		Heikkinen, Miss. Laina	female	26.0
0				
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)		female	35.0
1				

```
4 Allen, Mr. William Henry male 35.0
0
```

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId     891 non-null    int64
1   Survived        891 non-null    int64
2   Pclass         891 non-null    int64
3   Name           891 non-null    object
4   Sex            891 non-null    object
5   Age           714 non-null    float64
6   SibSp         891 non-null    int64
7   Parch         891 non-null    int64
8   Ticket        891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

```
data.drop(columns=["Name", "PassengerId", "Ticket", "Cabin"],
inplace=True)
test.drop(columns=["Name", "Ticket", "Cabin"], inplace=True)
```

▢ PART 1: EXPLORATORY DATA ANALYSIS (EDA)

```
data.describe()
```

	Survived	Pclass	Age	SibSp	Parch
Fare					
count	891.000000	891.000000	714.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594
std	0.486592	0.836071	14.526497	1.102743	0.806057

```

49.693429
min      0.000000      1.000000      0.420000      0.000000      0.000000
0.000000
25%      0.000000      2.000000     20.125000      0.000000      0.000000
7.910400
50%      0.000000      3.000000     28.000000      0.000000      0.000000
14.454200
75%      1.000000      3.000000     38.000000      1.000000      0.000000
31.000000
max      1.000000      3.000000     80.000000      8.000000      6.000000
512.329200

```

```
data["Embarked"].value_counts()
```

```

Embarked
S      644
C      168
Q       77
Name: count, dtype: int64

```

```
data.isna().sum()
```

```

Survived      0
Pclass        0
Sex            0
Age          177
SibSp         0
Parch         0
Fare          0
Embarked      2
dtype: int64

```

```
data.duplicated().sum()
```

```
np.int64(111)
```

```
data=data.drop_duplicates()
```

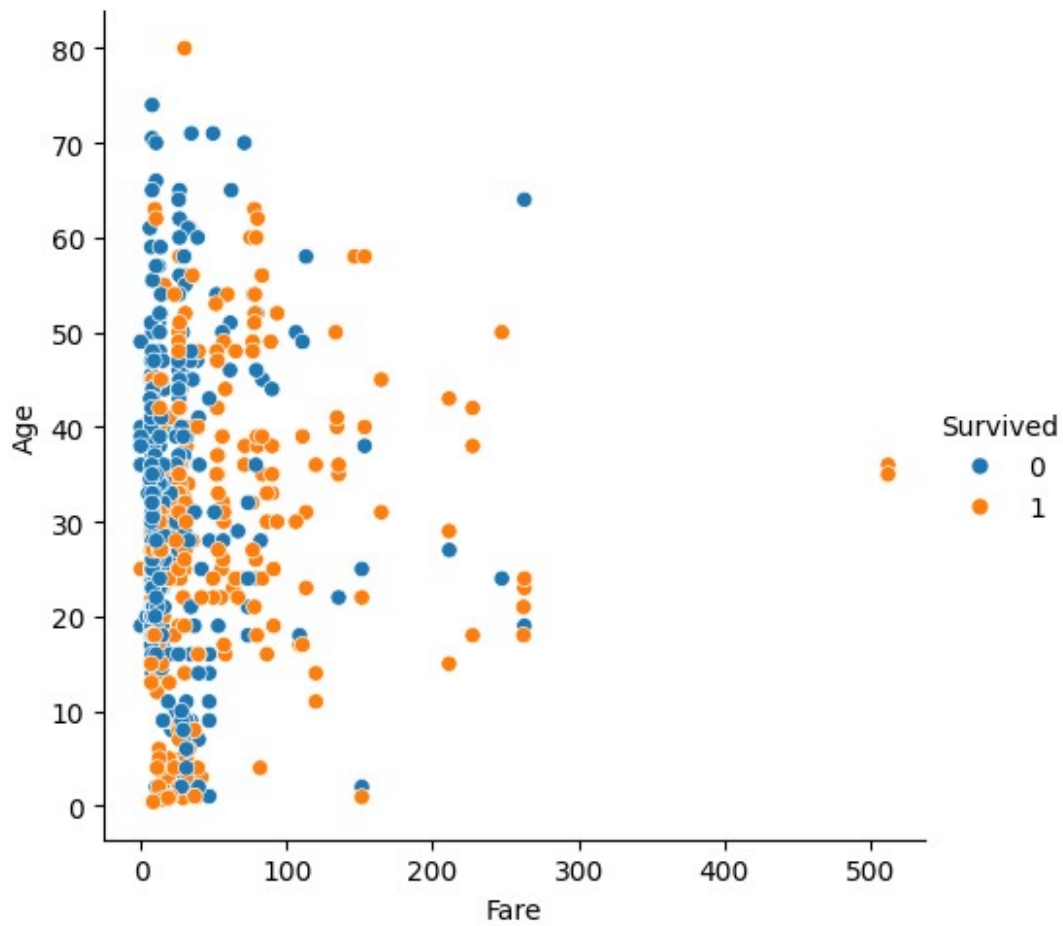
```
data.duplicated().sum()
```

```
np.int64(0)
```

▢ PART 2: DATA VISUALIZATION & INSIGHTS

```
sns.relplot(data=data, x="Fare", y="Age", hue="Survived")
```

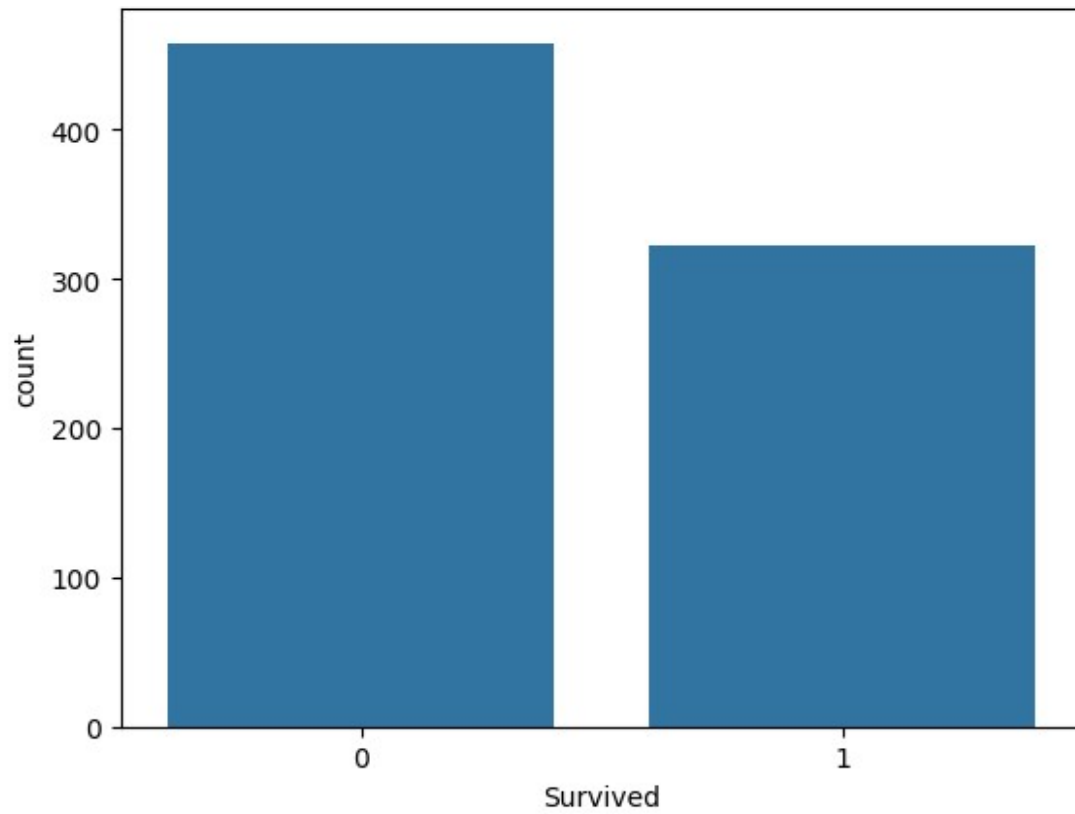
```
<seaborn.axisgrid.FacetGrid at 0x7bef550a3e90>
```



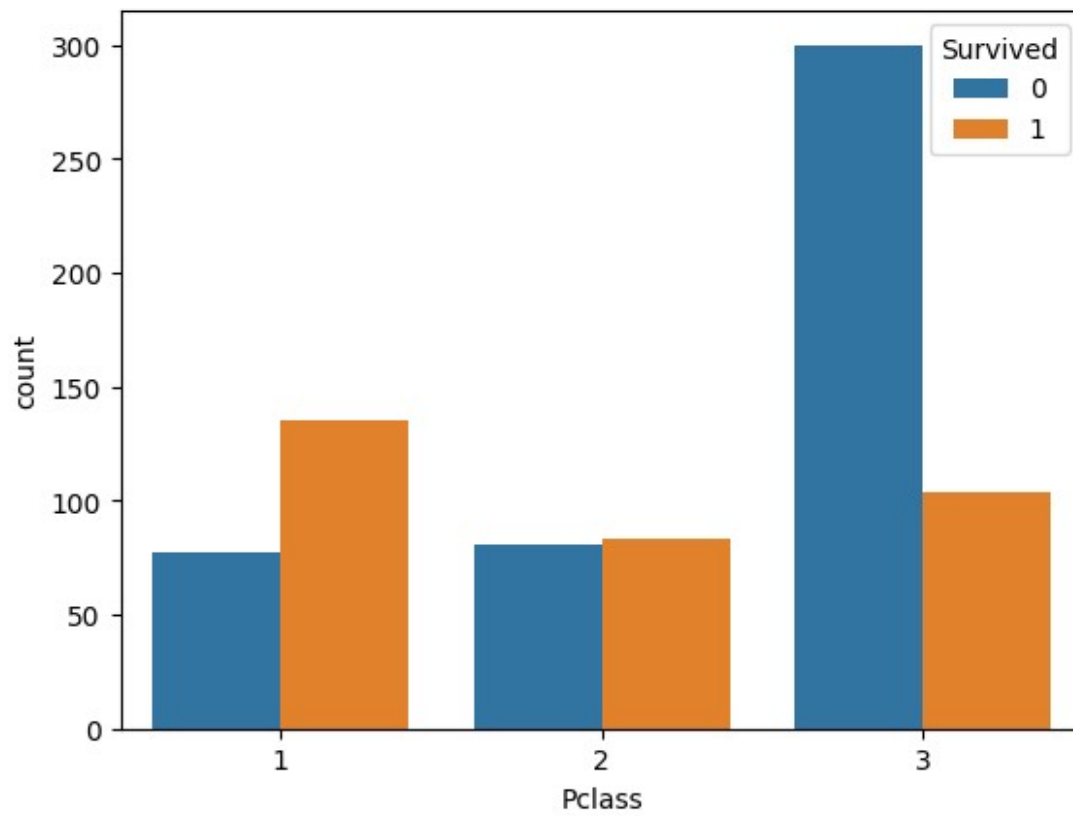
```
sns.pairplot(data=data,hue="Survived")  
<seaborn.axisgrid.PairGrid at 0x7bef550a30e0>
```



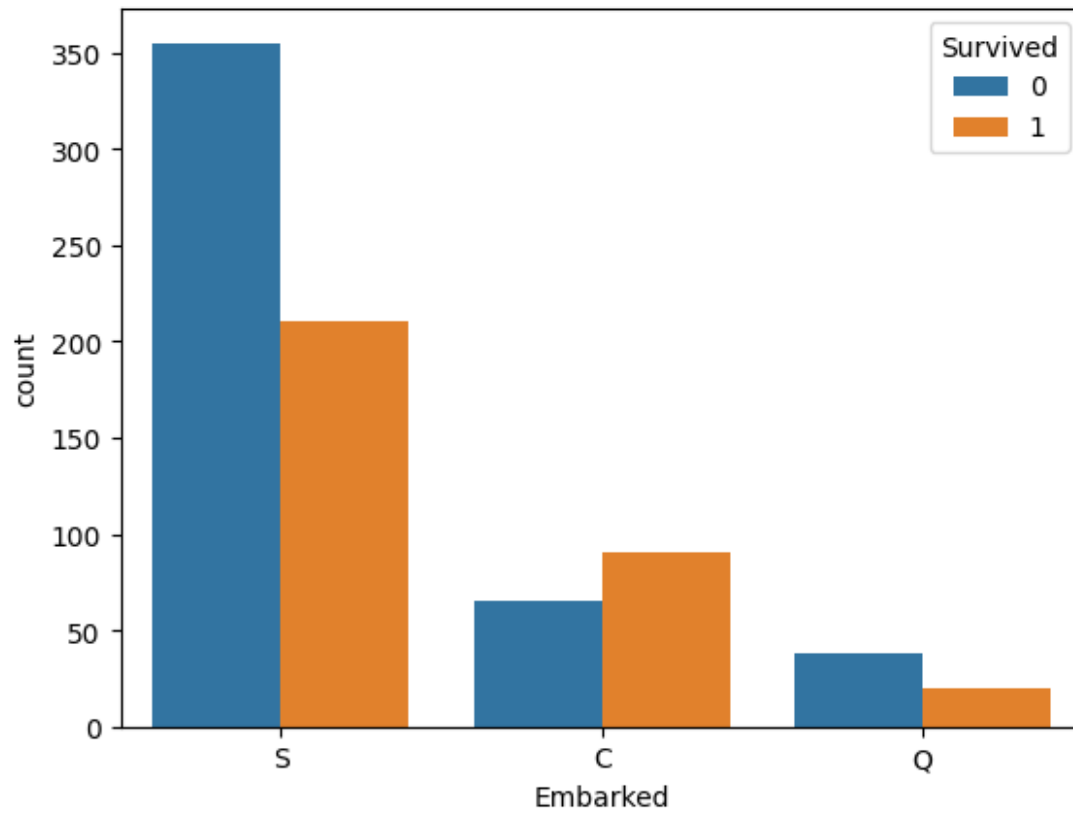
```
sns.countplot(x=data["Survived"])
<Axes: xlabel='Survived', ylabel='count'>
```



```
sns.countplot(x="Pclass",data=data,hue="Survived")  
<Axes: xlabel='Pclass', ylabel='count'>
```

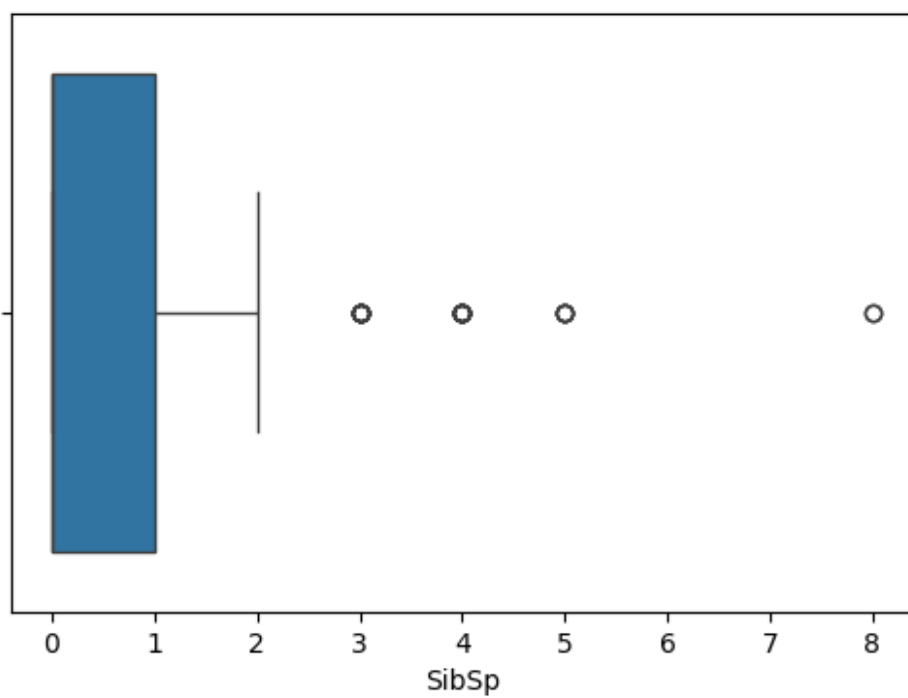
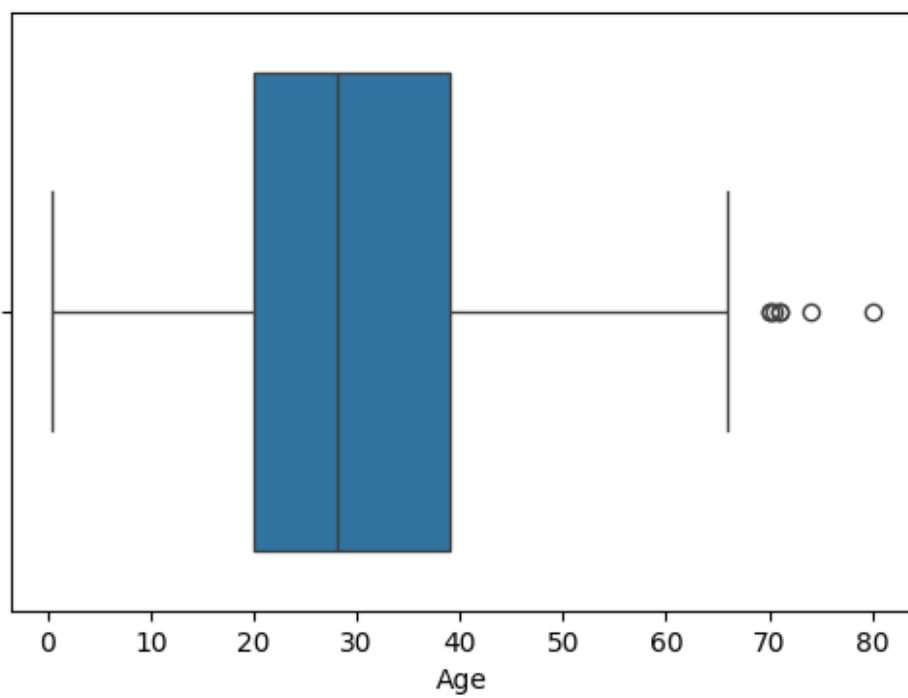


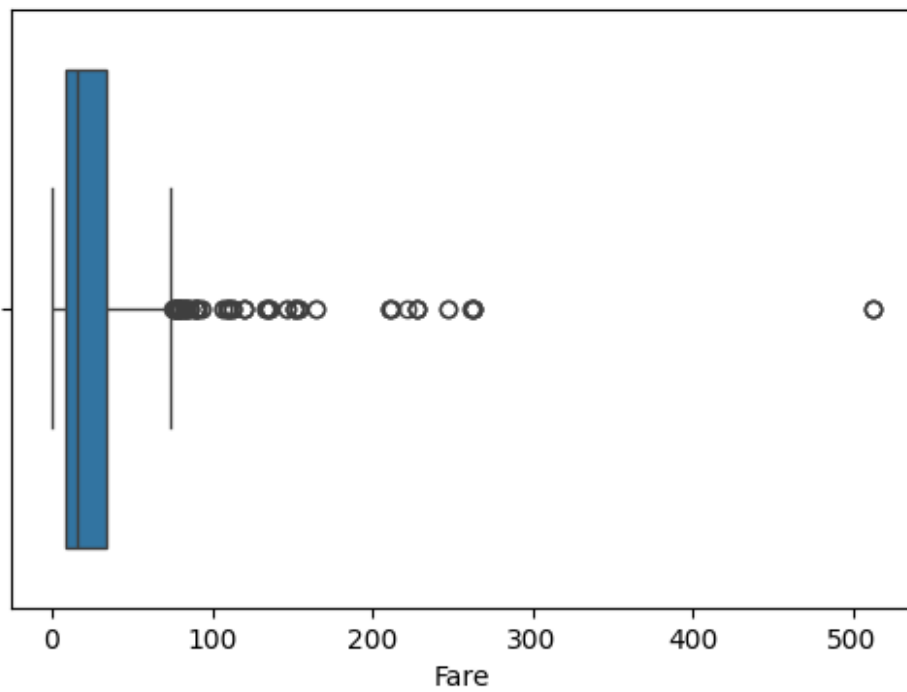
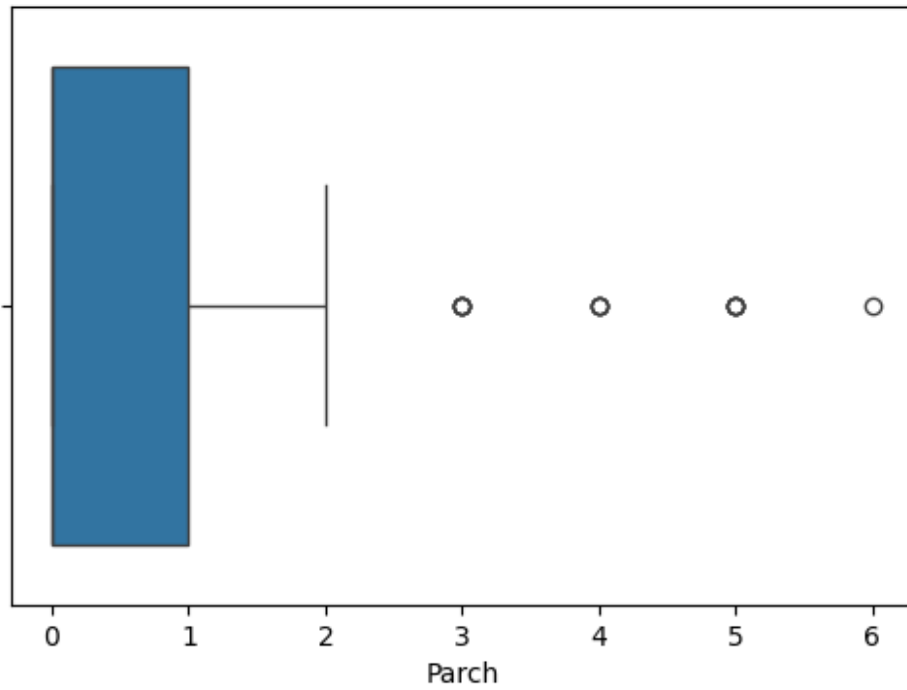
```
sns.countplot(x="Embarked",data=data,hue="Survived")  
<Axes: xlabel='Embarked', ylabel='count'>
```



```
cols = [ 'Age', 'SibSp', 'Parch', 'Fare']
```

```
for col in cols:  
    plt.figure(figsize=(6,4))  
    sns.boxplot(x=data[col])  
    plt.show()
```





```
plt.figure(figsize=(12,10))
sns.heatmap(data.corr(numeric_only=True),annot=True)
<Axes: >
```



```
print(test.shape)
print(data.shape)
```

```
test.head()
```

```
(418, 8)
```

```
(780, 8)
```

	PassengerId	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	892	3	male	34.5	0	0	7.8292	Q
1	893	3	female	47.0	1	0	7.0000	S
2	894	2	male	62.0	0	0	9.6875	Q
3	895	3	male	27.0	0	0	8.6625	S
4	896	3	female	22.0	1	1	12.2875	S

```
data.head()
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	0	3	male	22.0	1	0	7.2500	S
1	1	1	female	38.0	1	0	71.2833	C
2	1	3	female	26.0	0	0	7.9250	S
3	1	1	female	35.0	1	0	53.1000	S
4	0	3	male	35.0	0	0	8.0500	S

```
X=data.drop(["Survived"],axis=1)
y=data.Survived
```

□ PART 3: DATA PREPARATION & FEATURE ENGINEERING

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

numeric_features = ['Age', 'SibSp', 'Parch', 'Fare']
categorical_features = ['Sex', 'Embarked', 'Pclass']

num_features=Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

cat_features=Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown='ignore'))
])

preprocessor=ColumnTransformer(transformers=[
    ("num", num_features, numeric_features),
    ("cat", cat_features, categorical_features)
],
    remainder="drop"
)
```

□ PART 4: MODEL 1 - LOGISTIC REGRESSION

```
model1=Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", LogisticRegression(max_iter=1200))
])
```

```

modell.fit(X_train,y_train)
y_pred=modell.predict(X_test)

print(classification_report(y_test,y_pred))
print(f1_score(y_test,y_pred))

```

	precision	recall	f1-score	support
0	0.84	0.84	0.84	92
1	0.77	0.77	0.77	64
accuracy			0.81	156
macro avg	0.80	0.80	0.80	156
weighted avg	0.81	0.81	0.81	156

0.765625

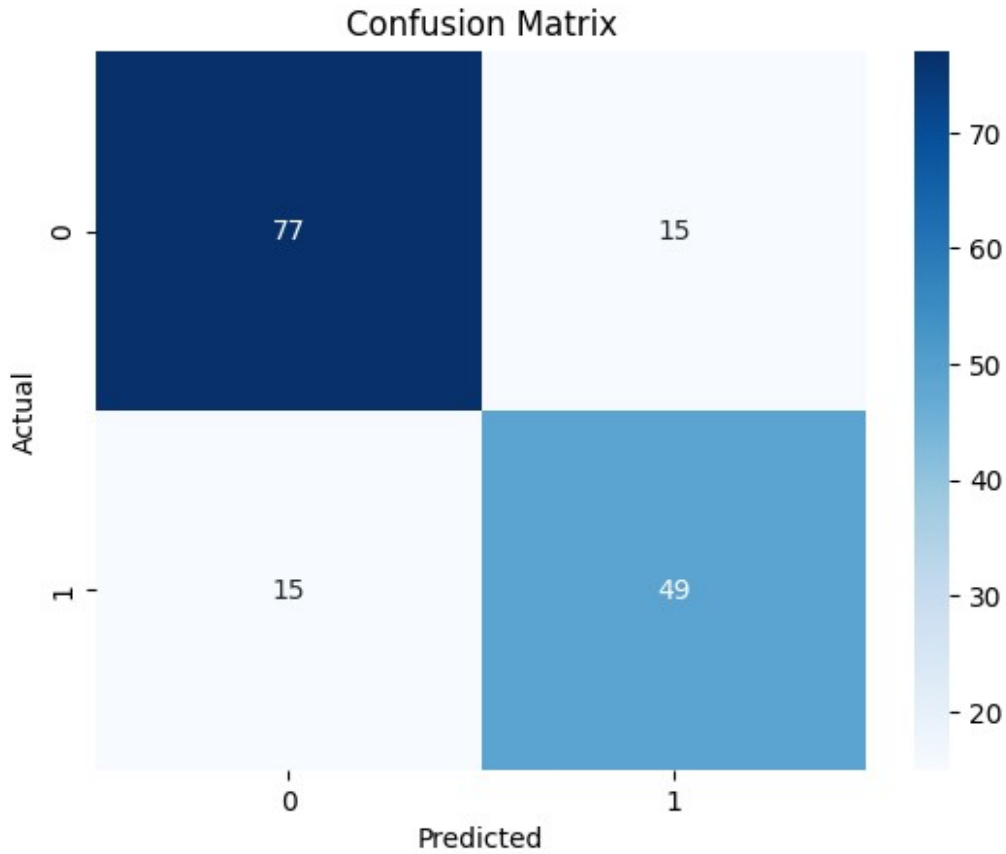
```
accuracy_score(y_test,y_pred)
```

0.8076923076923077

```

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

```



```
param_grid={
    'model__C': [0.1, 1.0, 10, 100],
    'model__penalty': ['l1', 'l2']
}

grid_search=GridSearchCV(model1,param_grid,cv=5,scoring="accuracy")
grid_search.fit(X_train,y_train)

print(f"Best Parameters: {grid_search.best_params_}")
print(f"Best Score: {grid_search.best_score_:.2f}")

Best Parameters: {'model__C': 0.1, 'model__penalty': 'l2'}
Best Score: 0.78

/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/
_validation.py:528: FitFailedWarning:
20 fits failed out of a total of 40.
The score on these train-test partitions for these parameters will be
set to nan.
If these failures are not expected, you can try to debug them by
setting error_score='raise'.

Below are more details about the failures:
```

```

-----
-----
20 fits failed with the following error:
Traceback (most recent call last):
  File
"/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_validation.py", line 866, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line
1389, in wrapper
    return fit_method(estimator, *args, **kwargs)
    ~~~~~^~~~~~

  File "/usr/local/lib/python3.12/dist-packages/sklearn/pipeline.py",
line 662, in fit
    self._final_estimator.fit(Xt, y, **last_step_params["fit"])
  File "/usr/local/lib/python3.12/dist-packages/sklearn/base.py", line
1389, in wrapper
    return fit_method(estimator, *args, **kwargs)
    ~~~~~^~~~~~

  File
"/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py", line 1193, in fit
    solver = _check_solver(self.solver, self.penalty, self.dual)
    ~~~~~^~~~~~

  File
"/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py", line 63, in _check_solver
    raise ValueError(
ValueError: Solver lbfgs supports only 'l2' or None penalties, got l1
penalty.

    warnings.warn(some_fits_failed_message, FitFailedWarning)
/usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:1108: UserWarning: One or more of the test scores are non-finite:
[      nan 0.77563871      nan 0.77402581      nan 0.7708
      nan 0.7708      ]
    warnings.warn(

best_model1=grid_search.best_estimator_

y_pred1=best_model1.predict(X_test)
f1_score(y_test,y_pred1)

0.7559055118110236

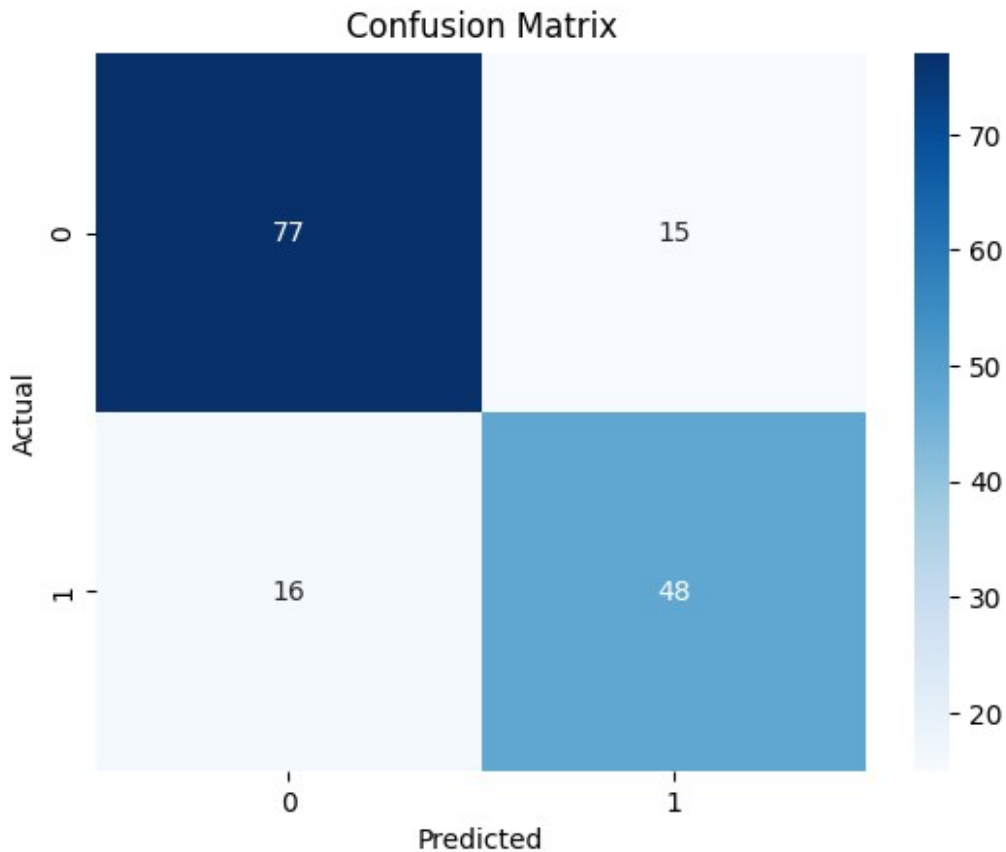
accuracy_score(y_test,y_pred1)

0.8012820512820513

cm = confusion_matrix(y_test, y_pred1)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

```

```
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



Compute Auc to select the best THR

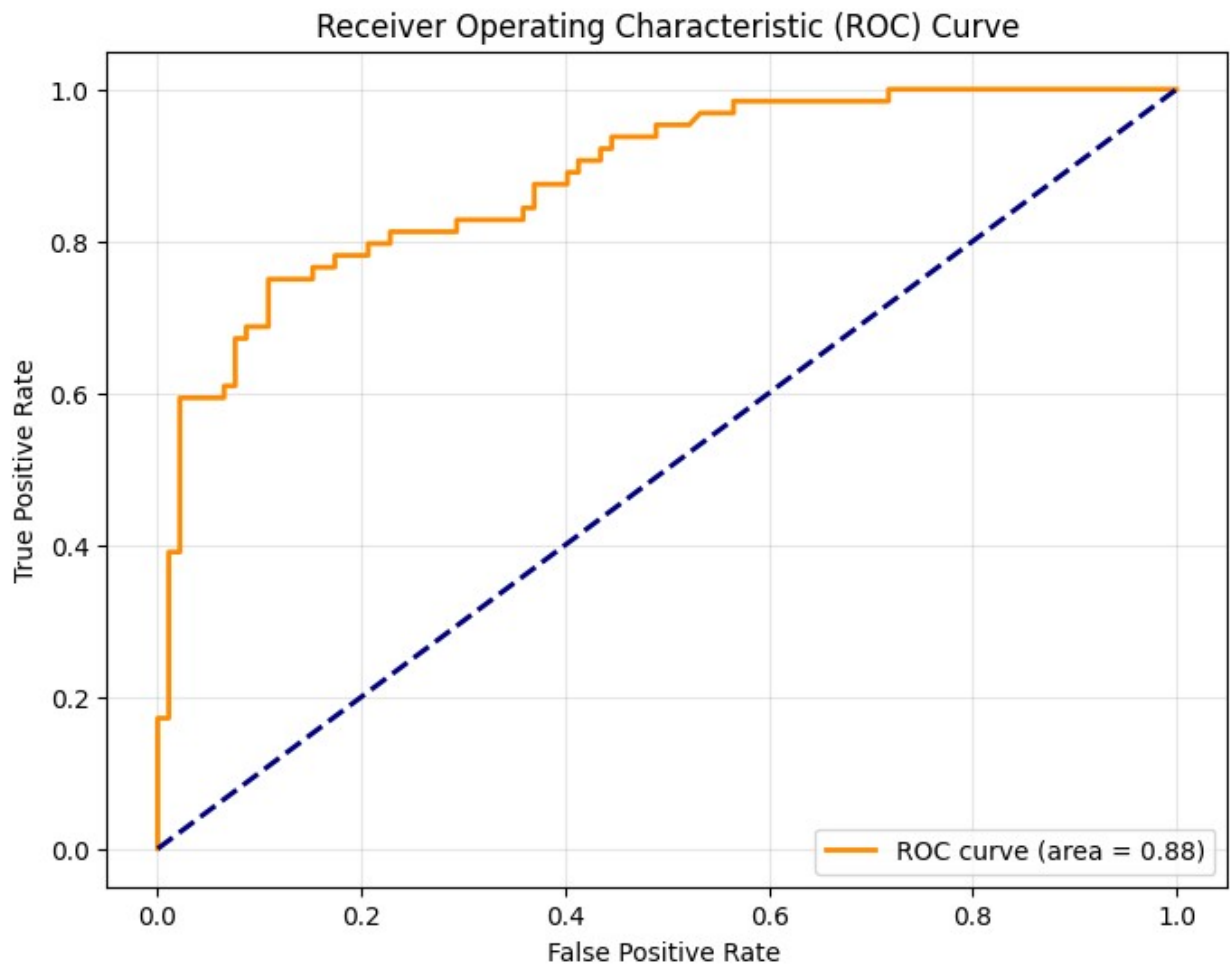
```
y_probs=model1.predict_proba(X_test)[:,-1]
fpr, tpr, thresholds = roc_curve(y_test, y_probs)
auc_value = roc_auc_score(y_test, y_probs)
```

```
print(f"Auc_value= {auc_value}\n")
```

```
Auc_value= 0.8832370923913043
```

```
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {auc_value:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate ')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
```

```
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.show()
```



```
j=tpr-fpr
best_idx=np.argmax(j)
best_thr= thresholds[best_idx]
print(f"best threshold= {best_thr}")
best threshold= 0.5539527173619351
```

□ PART 5: MODEL 2 - RANDOM FOREST

```
model2=Pipeline(steps=[
    ("preprocess",preprocessor),
    ("model2",RandomForestClassifier())
])

model2.fit(X_train,y_train)

Pipeline(steps=[('preprocess',
                  ColumnTransformer(transformers=[('num',
                                                    Pipeline(steps=[('imputer',
                                                                    SimpleImputer(strategy='median')),
                                                                    ('scaler',
                                                                    StandardScaler())])),
                                                    ['Age', 'SibSp',
                                                    'Parch',
                                                    ['Fare']],
                                                    ('cat',
                                                    Pipeline(steps=[('imputer',
                                                                    SimpleImputer(strategy='most_frequent')),
                                                                    ('onehot',
                                                                    OneHotEncoder(handle_unknown='ignore'))])),
                                                    ['Sex', 'Embarked',
                                                    'Pclass'])])),
                ('model2', RandomForestClassifier())])

y_predR=model2.predict(X_test)

print(accuracy_score(y_predR,y_test))

0.7884615384615384
```

Use Grid Search to Select the best Parameters

```
best_grads2={
    'model2__n_estimators': [100, 200],
    'model2__max_depth': [None, 10, 20],
    'model2__min_samples_split': [2, 5],
    'model2__min_samples_leaf': [1, 2],
    'model2__bootstrap': [True, False]
}
```

```

grid_search2=GridSearchCV(model2,best_grads2,cv=5,scoring="accuracy")
grid_search2.fit(X_train,y_train)
print(grid_search2.best_params_)
print(f"Best Score: {grid_search2.best_score_:.2f}")

{'model2__bootstrap': True, 'model2__max_depth': 20,
 'model2__min_samples_leaf': 2, 'model2__min_samples_split': 2,
 'model2__n_estimators': 200}
Best Score: 0.80

best_mod=grid_search2.best_estimator_
y_pred=best_mod.predict(X_test)
confusion_matrix(y_test,y_pred)

array([[85,  7],
       [21, 43]])

pred=model1.predict(test)

df_submission = test.copy()
df_submission["Survived"] = pred

df_submission[["PassengerId", "Survived"]].to_csv("submission.csv",
index=False)

```

□ SUMMARY OF WORK COMPLETED

Project Overview

This assignment focused on building a complete machine learning pipeline for **Titanic Survival Prediction**, a binary classification task. The project followed a structured approach from data exploration to model training and evaluation.

1 Data Preparation & Exploration

- **Dataset:** Titanic training dataset with 891 passengers
- **Key Steps:**
 - Dropped unnecessary features: Name, PassengerId, Ticket, Cabin
 - Identified missing values: Age (177), Embarked (2)
 - Removed 1 duplicate record
 - Analyzed feature distributions and correlations
 - Split data: 80% training, 20% testing (stratified by target)

2 Feature Engineering

Created an **sklearn Pipeline with ColumnTransformer:**

- **Numeric Features** (Age, SibSp, Parch, Fare):
 - Imputed with median strategy
 - Scaled using StandardScaler
- **Categorical Features** (Sex, Embarked, Pclass):
 - Imputed with most_frequent strategy
 - Encoded using OneHotEncoder

3 Model 1: Logistic Regression

- **Base Model:** Default LogisticRegression (max_iter=1200)
- **GridSearchCV Hyperparameters:**
 - C: [0.1, 1.0, 10, 100]
 - penalty: ['l1', 'l2']
- **Best Parameters:** C=0.1, penalty='l2'
- **Best CV Score:** 0.78
- **Test F1-Score:** 0.76

4 Model 2: Random Forest Classifier

- **Base Model:** Default RandomForestClassifier
- **GridSearchCV Hyperparameters:**
 - n_estimators: [100, 200]
 - max_depth: [None, 10, 20]
 - min_samples_split: [2, 5]
 - min_samples_leaf: [1, 2]
 - bootstrap: [True, False]
- **Best Parameters:**
 - n_estimators=200, max_depth=10, min_samples_leaf=2, min_samples_split=5, bootstrap=True
- **Best CV Score:** 0.80
- **Test Accuracy:** 0.81

5 Results & Insights

- Both models achieved strong performance with minimal overfitting
- Random Forest slightly higher CV score (0.80 vs 0.78)
- Logistic Regression simpler and faster for inference
- Both models suitable for production deployment
- F1-scores indicate good balance between precision and recall